

Webhooks

CVAT Webhooks: set up and use

Webhooks are user-defined HTTP callbacks that are triggered by specific events. When an event that triggers a webhook occurs, CVAT makes an HTTP request to the URL configured for the webhook. The request will include a payload with information about the event.

CVAT, webhooks can be triggered by a variety of events, such as the creation, deletion, or modification of tasks, jobs, and so on. This makes it easy to set up automated processes that respond to changes made in CVAT.

For example, you can set up webhooks to alert you when a job's assignee is changed or when a job/task's status is updated, for instance, when a job is completed and ready for review or has been reviewed. New task creation can also trigger notifications.

These capabilities allow you to keep track of progress and changes in your CVAT workflow instantly.

In CVAT you can create a webhook for a project or organization. You can use CVAT GUI or direct API calls.

See:

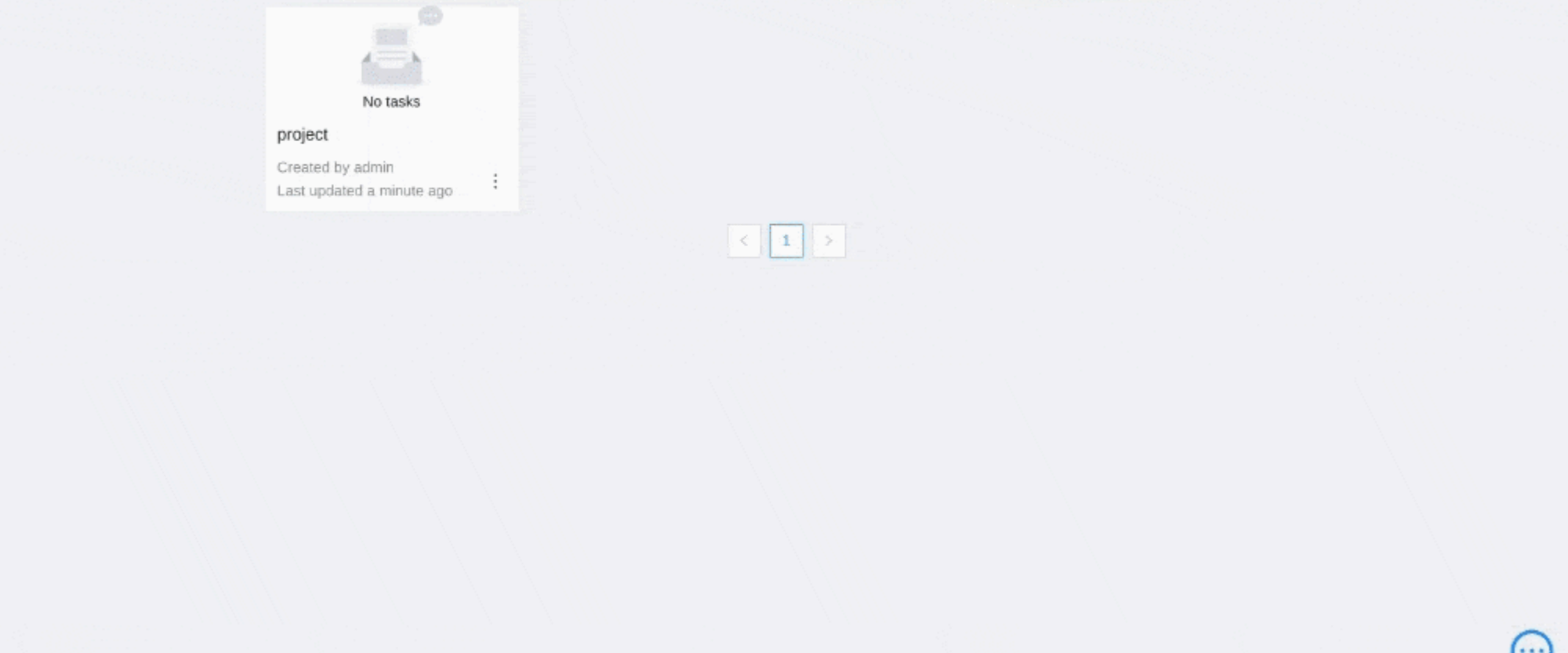
- [Create Webhook](#)
 - [For project](#)
 - [For organization](#)
 - [Webhooks forms](#)
 - [List of events](#)
- [Payloads](#)
 - [Create event](#)
 - [Update event](#)
 - [Delete event](#)
- [Webhook secret](#)
- [Ping Webhook](#)
- [Webhooks with API calls](#)
- [Example of setup and use](#)

Create Webhook

For project

To create a webhook for **Project**, do the following:

1. [Create a Project](#).
2. Go to the **Projects** and click on the project's widget.
3. In the top right corner, click **Actions** > **Setup Webhooks**.
4. In the top right corner click +

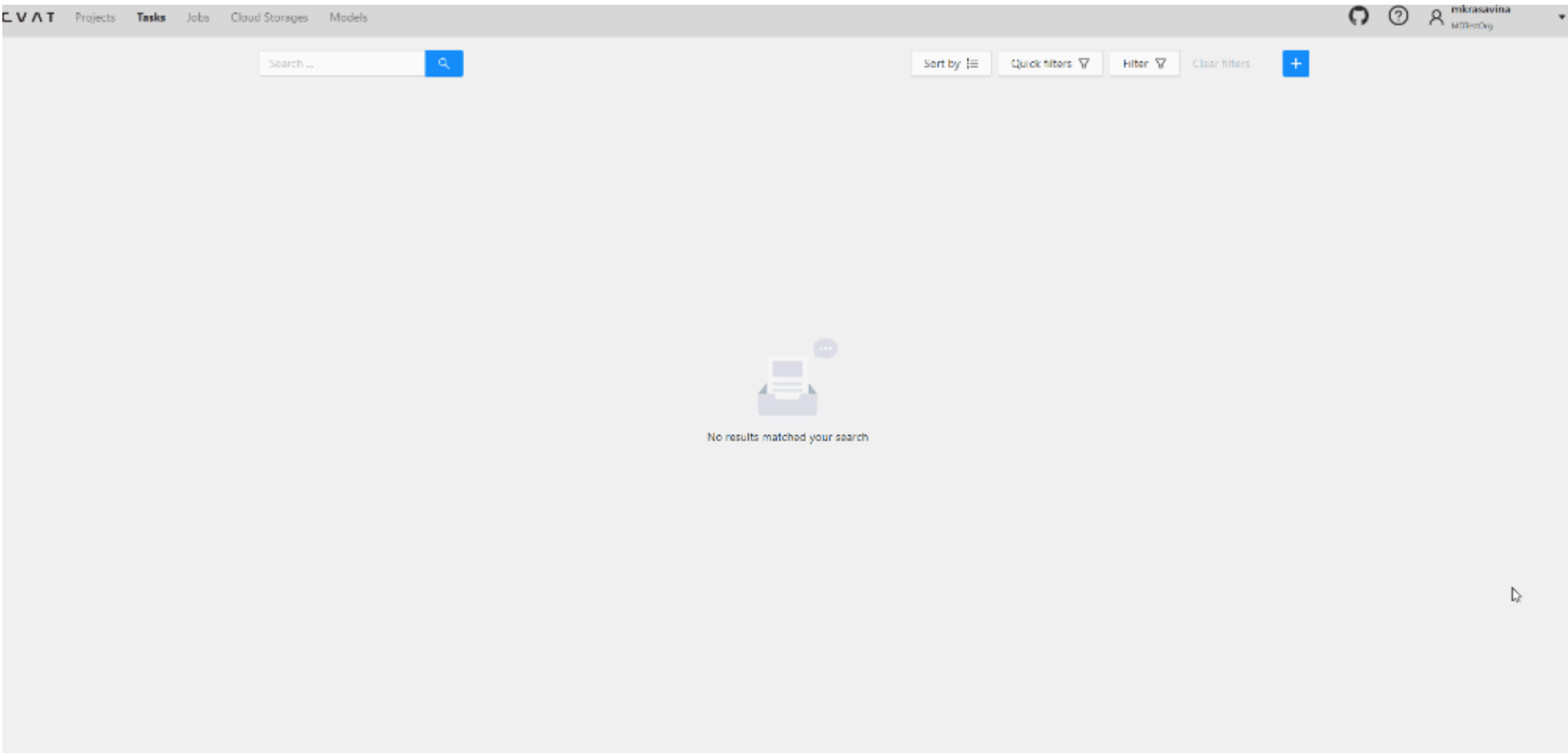


5. Fill in the [Setup webhook](#) form and click **Submit**.

For organization

To create a webhook for **Organization**, do the following:

1. [Create Organization](#)
2. Go to the **Organization** > **Settings** > **Actions** > **Setup Webhooks**.
3. In the top right corner click +



4. Fill in the [Setup webhook](#) form and click **Submit**.

Webhooks forms

The **Setup a webhook** forms look like the following.

Description

Project

Webhooks Demo

Content type

application/json

Secret

☒ Enable SSL

Verify SSL certificates when delivering payloads

☒ Active

CVAT will deliver events for active webhooks only

☐ Send everything

☒ Select individual events

☐ comment

☐ issue

☐ job

☐ task

☐ project

Submit

Description

Project

Select project

Content type

application/json

Secret

☒ Enable SSL

Verify SSL certificates when delivering payloads

☒ Active

CVAT will deliver events for active webhooks only

☐ Send everything

☒ Select individual events

☐ comment

☐ invitation

☐ issue

☐ job

☐ membership

☐ project

☐ task

☐ organization

Submit

Project Webhook form

Organization Webhook form

Forms have the following fields:

Field	Description
Target URL	The URL where the event data will be sent.
Description	Provides a brief summary of the webhook's purpose.
Project	A drop-down list that lets you select from available projects.
Content type	Defines the data type for the payload in the webhook request via the HTTP Content-Type field.
Secret	A unique key for verifying the webhook's origin, ensuring it's genuinely from CVAT. For more information, see Webhook secret
Enable SSL	A checkbox for enabling or disabling SSL verification .
Active	Uncheck this box if you want to stop the delivery of specific webhook payloads.
Send everything	Check this box to send all event types through the webhook.
Specify individual events	Choose this option to send only certain event types. Refer to the List of available events for more information on event types.

List of events

The following events are available for webhook alerts.

Resource	Create	Update	Delete	Description
Organization		✔		Alerts for changes made to an Organization.
Membership		✔	✔	Alerts when a member is added to or removed from an organization.
Invitation	✔		✔	Alerts when an invitation to an Organization is issued or revoked.
Project	✔	✔	✔	Alerts for any actions taken within a project.

task	✓	✓	✓	Alerts for actions related to a task, such as status changes, assignments, etc.
Job		✓		Alerts for any updates made to a job.
Issue	✓	✓	✓	Alerts for any activities involving issues.
Comment	✓	✓	✓	Alerts for actions involving comments, such as creation, deletion, or modification.

Payloads

Create event

Webhook payload object for `create:<resource>` events:

Key	Type	Description
event	string	Identifies the event that triggered the webhook, following the <code>create:<resource></code> pattern.
<resource>	object	Complete information about the created resource. Refer to the Swagger docs for individual resource details.
webhook_id	integer	The identifier for the webhook that sends the payload.
sender	object	Details about the user that triggered the webhook.

An example of payload for the `create:task` event:

```
{
  "event": "create:task",
  "task": {
    "url": "<http://localhost:8080/api/tasks/15>",
    "id": 15,
    "name": "task",
    "project_id": 7,
    "mode": "",
    "owner": {
      "url": "<http://localhost:8080/api/users/1>",
      "id": 1,
      "username": "admin1",
      "first_name": "Admin",
      "last_name": "First"
    },
    "assignee": null,
    "bug_tracker": "",
    "created date": "2022-10-04T08:05:50.419259Z",
```

Copy

Update event

Webhook payload object for `update:<resource>` events:

Key	Type	Description
event	string	Identifies the event that triggered the webhook, following the <code>update:<resource></code> pattern.
<resource>	object	Provides complete information about the updated resource. See the Swagger docs for resource details.
before_update	object	Contains keys of <code><resource></code> that were updated, along with their old values.
webhook_id	integer	The identifier for the webhook that dispatched the payload.

`sender` `object` Details about the user that triggered the webhook.

An example of `update:<resource>` event:

Copy

```
{
  "event": "update:task",
  "task": {
    "url": "<http://localhost:8080/api/tasks/15>",
    "id": 15,
    "name": "new task name",
    "project_id": 7,
    "mode": "annotation",
    "owner": {
      "url": "<http://localhost:8080/api/users/1>",
      "id": 1,
      "username": "admin1",
      "first_name": "Admin",
      "last_name": "First"
    },
    "assignee": null,
    "bug_tracker": "",
    "created date": "2022-10-04T08:05:50.419259Z",
```

Delete event

Webhook payload object for `delete:<resource>` events:

Key	Type	Description
<code>event</code>	<code>string</code>	Identifies the event that triggered the webhook, following the <code>delete:<resource></code> pattern.
<code><resource></code>	<code>object</code>	Provides complete information about the deleted resource. See the Swagger docs for resource details.
<code>webhook_id</code>	<code>integer</code>	The identifier for the webhook that dispatched the payload.
<code>sender</code>	<code>object</code>	Details about the user that triggered the webhook.

Here is an example of the payload for the `delete:task` event:

```
{
  "event": "delete:task",
  "task": {
    "url": "<http://localhost:8080/api/tasks/15>",
    "id": 15,
    "name": "task",
    "project_id": 7,
    "mode": "",
    "owner": {
      "url": "<http://localhost:8080/api/users/1>",
      "id": 1,
      "username": "admin1",
      "first_name": "Admin",
      "last_name": "First"
    },
    "assignee": null,
    "bug_tracker": "",
```

Webhook secret

To validate that the webhook requests originate from CVAT, include a `secret` during the webhook creation process.

When a `secret` is provided for the webhook, CVAT includes an `X-Signature-256` in the request header of the webhook.

CVAT uses the SHA256 hash function to encode the request body for the webhook and places the resulting hash into the header.

The webhook recipient can verify the source of the request by comparing the received `X-Signature-256` value with the expected value.

Here's an example of a header value for a request with an empty body and `secret = mykey` :

```
X-Signature-256: e1b24265bf2e0b20c81837993b4f1415f7b68c503114d100a40601eca6a2745f
```

Here is an example of how you can verify a webhook signature in your webhook receiver service:

```
# webhook_receiver.py

import hmac
from hashlib import sha256
from flask import Flask, request

app = Flask(__name__)

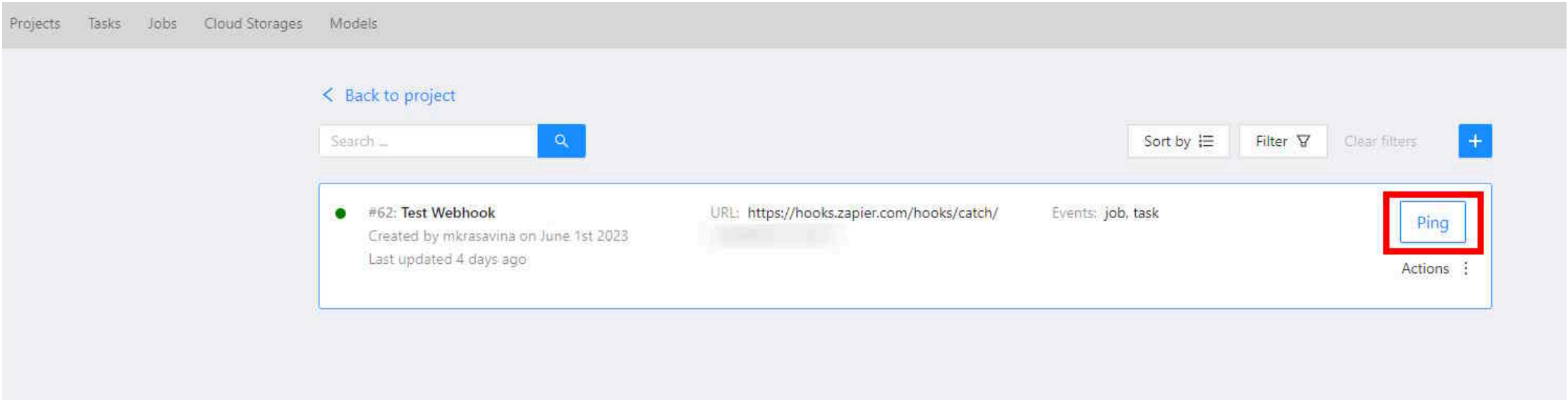
@app.route("/webhook", methods=["POST"])
def webhook():
    signature = (
        "sha256="
        + hmac.new("mykey".encode("utf-8"), request.data, digestmod=sha256).hexdigest()
    )

    if hmac.compare_digest(request.headers["X-Signature-256"], signature):
        return app.response_class(status=200)

    raise app.response_class(status=500, response="Signatures didn't match!")
```

Ping Webhook

To confirm the proper configuration of your webhook and ensure that CVAT can establish a connection with the target URL, use the **Ping** webhook feature.



1. Click the **Ping** button in the user interface (or send a `POST /webhooks/{id}/ping` request through API).
2. CVAT will send a webhook alert to the specified target URL with basic information about the webhook.

Ping webhook payload:

event	string	The value is always <code>ping</code> .
webhook	object	Complete information about the webhook. See the Swagger docs for a detailed description of fields.
sender	object	Information about the user who initiated the <code>ping</code> on the webhook.

Here is an example of a payload for the `ping` event:

```
{
  "event": "ping",
  "webhook": {
    "id": 7,
    "url": "<http://localhost:8080/api/webhooks/7>",
    "target_url": "<https://example.com>",
    "description": "",
    "type": "project",
    "content_type": "application/json",
    "is_active": true,
    "enable_ssl": true,
    "created_date": "2022-10-04T08:05:23.007381Z",
    "updated_date": "2022-10-04T08:05:23.007395Z",
    "owner": {
      "url": "<http://localhost:8080/api/users/1>",
      "id": 1,
      "username": "admin1",
      "first name": "Admin",
```

Copy

Webhooks with API calls

To create webhook via an API call, see [Swagger documentation](#).

For examples, see [REST API tests](#).

Example of setup and use

This video demonstrates setting up email alerts for a project using Zapier and Gmail.

